

Enterprise Real-Time Data Platform: Case Study

Name: Madhu M M

1. Project Overview:

This project focuses on building an enterprise-grade, end-to-end real-time data platform designed to process synthetic financial transaction data. The solution begins by streaming high-velocity transactional events from a Kafka or Azure Event Hub producer into a Delta Lake Bronze layer, capturing raw, immutable records for audit and replay. The ingested data is then validated and profiled using **Great Expectations**, ensuring data quality and consistency before being transformed into curated Silver tables. Business logic, enrichment, and advanced data masking are applied in the Gold layer to make the data analytics-ready and privacy-compliant. Additionally, **Apache NiFi** is integrated into the architecture to support dynamic transformations, PII masking, and external system interfacing. The dataset includes key transactional attributes such as `transaction_id`, `customer_id`, `amount`, `date`, `time`, `location`, `cardtype`, `cardnumber`, `payment_status`, and `is_fraud`, supporting use cases like fraud detection, payment analytics, and customer behavior insights.

2. continuous streaming data generation

```
File Edit Selection View Go Run ... case-study
pos_transactions_producer.py X
pos_transactions_producer.py > ...
1 import json, random
2 import time as t
3 from datetime import date, timedelta, time
4
5 from kafka import KafkaProducer
6
7 producer = KafkaProducer(
8     bootstrap_servers='madhu-kafka-namespace.servicebus.windows.net:9093',
9     security_protocol='SASL_SSL',
10    sasl_mechanism='PLAIN',
11    sasl_plain_username='${ConnectionString}',
12    sasl_plain_password='Endpoint=sb://madhu-kafka-namespace.servicebus.windows.net;/SharedAccessKeyName=RootManageSharedAccessKey;SharedAccessKey=
13    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
14    key_serializer=lambda k: k.encode('utf-8')
15 )
16
17
18 users = [f"U{uid}" for uid in range(1, 10000)]
19
20 locations = ["Delhi", "Mumbai", "Bangalore", "Chennai", "Kolkata", "Hyderabad", "Jaipur", "Goa",
21             "Varanasi", "Leh", "Darjeeling", "Udaipur", "Mysore", "Shillong", "Kochi", "Mandya"]
22
23 card_types = ["Visa", "MasterCard", "RuPay", "American Express", "Diners Club"]
24
25 merchants = [
26     "Amazon", "Flipkart", "Malmart", "Target", "Best Buy",
27     "Starbucks", "McDonald's", "Shell", "ExxonMobil", "Apple Store"
28 ]
29
30 status = ["Success", "Success", "Success", "Success", "Success", "Success", "Failed"]
31 is_fraud = ["yes", "no", "no", "no", "no",]
32
```

```
azureruser@madhu-VM: ~
(kafka-env) azureruser@madhu-VM:~$ python3 pos_transactions_producer.py
Sending: {'transactionId': 'TX4044844', 'userId': 'U7802', 'cardNumber': '7304154287881958', 'amount': 555.62, 'location': 'Leh', 'date': '2020-05-20', 'time': '08:54:25', 'cardType': 'MasterCard', 'merchant': 'Shell', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX1681069', 'userId': 'U7751', 'cardNumber': '2967154784064264', 'amount': 1046.94, 'location': 'Shillong', 'date': '2025-02-22', 'time': '19:20:52', 'cardType': 'MasterCard', 'merchant': 'Target', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX9114618', 'userId': 'U3501', 'cardNumber': '3210884286482579', 'amount': 544.59, 'location': 'Hyderabad', 'date': '2022-05-17', 'time': '20:23:34', 'cardType': 'Visa', 'merchant': 'Best Buy', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX466820', 'userId': 'U8836', 'cardNumber': '9031658426942226', 'amount': 1558.48, 'location': 'Kochi', 'date': '2024-02-25', 'time': '03:34:37', 'cardType': 'MasterCard', 'merchant': 'McDonald's', 'payment_status': 'Failed', 'flag': 'no'}
Sending: {'transactionId': 'TX6516776', 'userId': 'U1047', 'cardNumber': '4811586658528473', 'amount': 58.63, 'location': 'Leh', 'date': '2023-05-23', 'time': '03:57:23', 'cardType': 'American Express', 'merchant': 'Starbucks', 'payment_status': 'Success', 'flag': 'yes'}
Sending: {'transactionId': 'TX2672930', 'userId': 'U2497', 'cardNumber': '9190173545992800', 'amount': 104.65, 'location': 'Udaipur', 'date': '2024-11-22', 'time': '00:59:07', 'cardType': 'Visa', 'merchant': 'ExxonMobil', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX8604053', 'userId': 'U5401', 'cardNumber': '2031927627936130', 'amount': 1357.52, 'location': 'Hyderabad', 'date': '2020-08-30', 'time': '07:01:50', 'cardType': 'Diners Club', 'merchant': 'Amazon', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX3477636', 'userId': 'U6446', 'cardNumber': '6410712091136706', 'amount': 732.12, 'location': 'Kolkata', 'date': '2025-02-16', 'time': '10:04:54', 'cardType': 'American Express', 'merchant': 'Starbucks', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX9257962', 'userId': 'U1881', 'cardNumber': '5201182097811483', 'amount': 808.73, 'location': 'Mumbai', 'date': '2018-05-07', 'time': '14:16:37', 'cardType': 'Visa', 'merchant': 'ExxonMobil', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX4954593', 'userId': 'U6125', 'cardNumber': '7182188393041970', 'amount': 332.1, 'location': 'Delhi', 'date': '2022-06-01', 'time': '13:51:05', 'cardType': 'RuPay', 'merchant': 'ExxonMobil', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX8487994', 'userId': 'U7098', 'cardNumber': '8133714010685674', 'amount': 1467.68, 'location': 'Jaipur', 'date': '2025-03-09', 'time': '20:52:16', 'cardType': 'Diners Club', 'merchant': 'Shell', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX3369314', 'userId': 'U5176', 'cardNumber': '7123869490511409', 'amount': 1508.03, 'location': 'Shillong', 'date': '2019-08-08', 'time': '01:23:56', 'cardType': 'Diners Club', 'merchant': 'Starbucks', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX4468034', 'userId': 'U9042', 'cardNumber': '8767383666549033', 'amount': 1509.64, 'location': 'Mysore', 'date': '2024-04-19', 'time': '16:30:13', 'cardType': 'MasterCard', 'merchant': 'Flipkart', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX1209383', 'userId': 'U6145', 'cardNumber': '3149282722419867', 'amount': 1763.57, 'location': 'Mysore', 'date': '2018-09-01', 'time': '14:34:42', 'cardType': 'MasterCard', 'merchant': 'Best Buy', 'payment_status': 'Success', 'flag': 'yes'}
Sending: {'transactionId': 'TX6925749', 'userId': 'U2964', 'cardNumber': '6051127347810508', 'amount': 1239.63, 'location': 'Leh', 'date': '2022-10-10', 'time': '03:50:28', 'cardType': 'Visa', 'merchant': 'Target', 'payment_status': 'Success', 'flag': 'no'}
Sending: {'transactionId': 'TX6170674', 'userId': 'U1762', 'cardNumber': '1613147381267613', 'amount': 235.15, 'location': 'Bangalore', 'date': '2019-02-25', 'time': '08:03:51', 'cardType': 'American Express', 'merchant': 'Flipkart', 'payment_status': 'Success', 'flag': 'no'}
```

Kafka Transaction Data Generator Script

This script simulates credit card transactions and streams them in real time to an Azure Event Hubs Kafka-compatible endpoint.

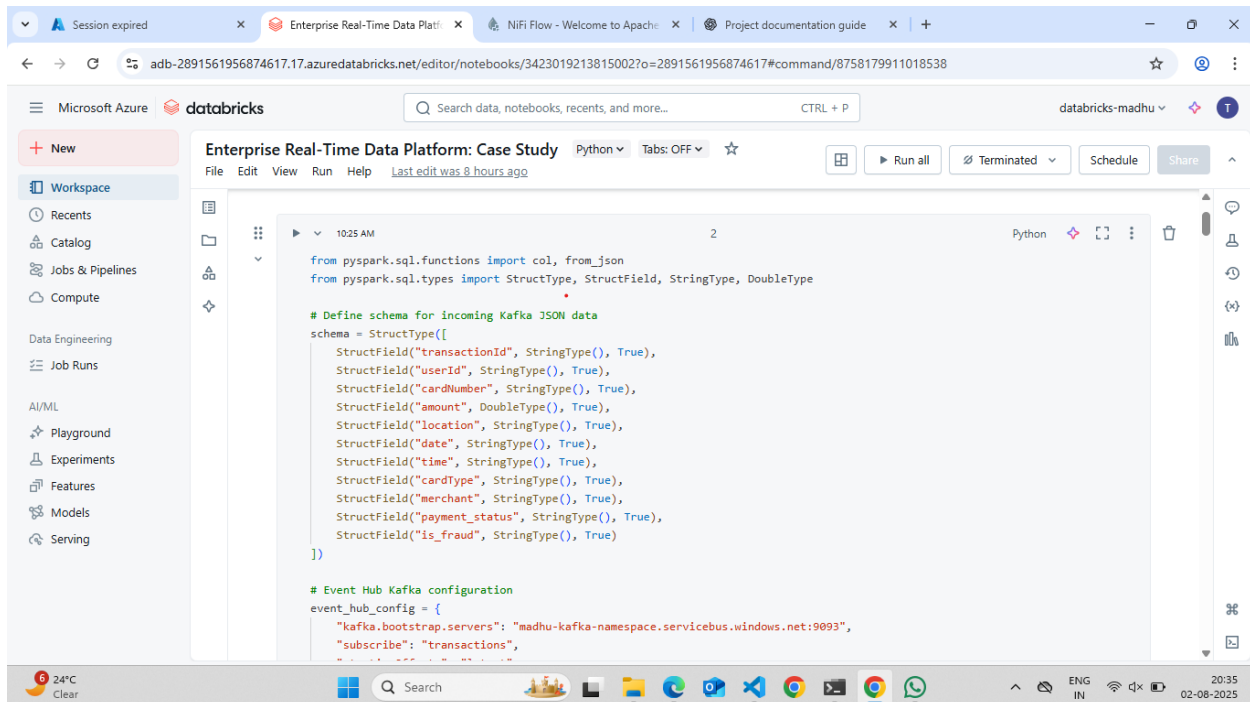
How It Works:

- Uses the KafkaProducer to connect to Azure Event Hubs using SASL_SSL authentication.
- Randomly generates:
 - User IDs, locations, card types, merchants, and transaction metadata
 - Transaction date & time
 - **Card number**
 - Payment status and fraud flag
- Sends this as a JSON message to the Kafka topic "transactions".

Output:

- A new transaction is generated every 10 milliseconds (0.01 sec).
- Printed to console and streamed to Kafka topic.

3. consumer:



The screenshot displays a Databricks workspace interface. The top navigation bar includes the Microsoft Azure logo, the Databricks logo, and a search bar. The main workspace area shows a notebook titled "Enterprise Real-Time Data Platform: Case Study" with a Python tab selected. The notebook content includes imports for PySpark SQL functions and types, a schema definition for incoming Kafka JSON data, and an Event Hub Kafka configuration. The schema defines fields for transactionId, userId, cardNumber, amount, location, date, time, cardType, merchant, payment_status, and is_fraud. The configuration specifies the bootstrap servers and the topic to subscribe to.

```
from pyspark.sql.functions import col, from_json
from pyspark.sql.types import StructType, StructField, StringType, DoubleType

# Define schema for incoming Kafka JSON data
schema = StructType([
    StructField("transactionId", StringType(), True),
    StructField("userId", StringType(), True),
    StructField("cardNumber", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("location", StringType(), True),
    StructField("date", StringType(), True),
    StructField("time", StringType(), True),
    StructField("cardType", StringType(), True),
    StructField("merchant", StringType(), True),
    StructField("payment_status", StringType(), True),
    StructField("is_fraud", StringType(), True)
])

# Event Hub Kafka configuration
event_hub_config = {
    "kafka.bootstrap.servers": "madhu-kafka-namespace.servicebus.windows.net:9093",
    "subscribe": "transactions",
}
```

Consumes the streaming transaction data coming from the Event Hub (configured as a Kafka-compatible source) and parses it into structured format for further processing.

4. Data Ingestion (Bronze)

Used Spark Structured Streaming to connect to the Event Hub (Kafka-compatible).

Configured `.readStream.format("kafka")` with Event Hub connection settings.

Defined a schema and used `from_json` to parse the incoming

JSON messages. Extracted required fields from the JSON value column.

Stored the raw streaming data into a Delta Lake table: `/delta_transactions`.

5. Data Validation on Bronze Layer (Great Expectations)

Connected Bronze Delta table using Spark to load the raw streaming data. Applied Great Expectations (GE) to check data quality on the ingested raw data.

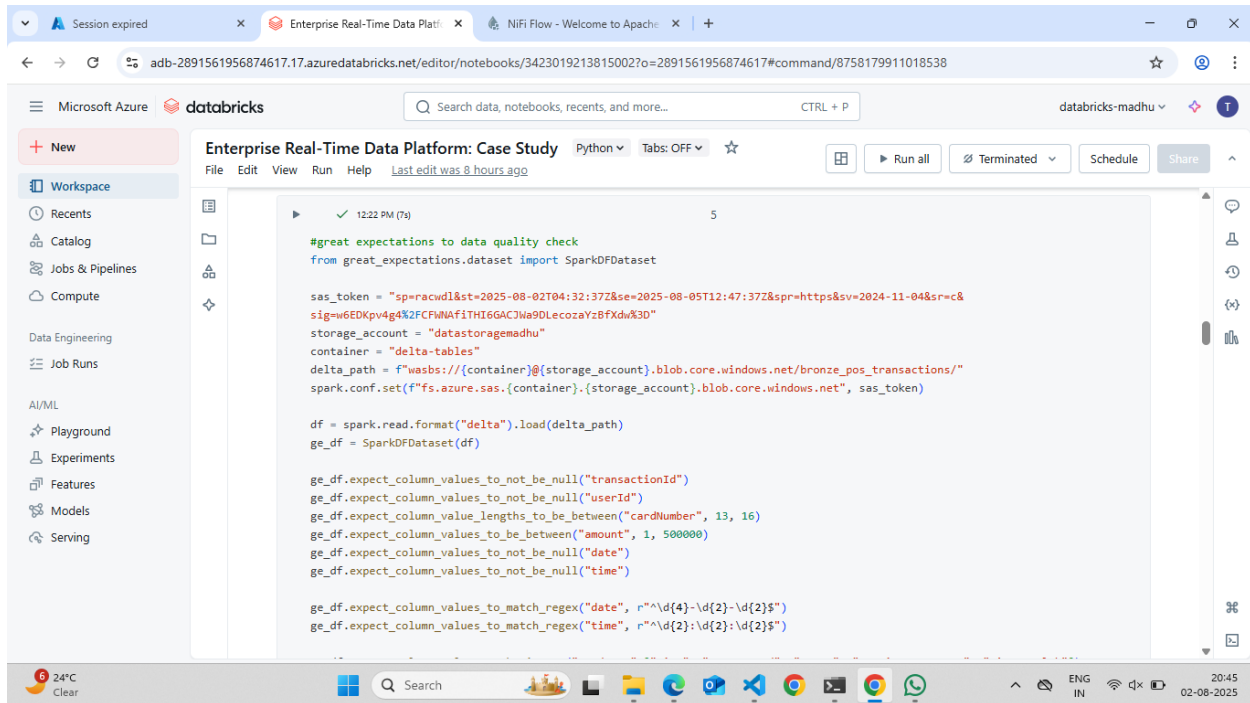
Defined rules:

Checked that `transaction_id` is not null and unique.

Verified `user_id`, `amount`, and `location` are not null.

Verified `cardnumber` length is exactly 16 digits.

Ran validation using `SparkDFDataset` and `validate()` method.

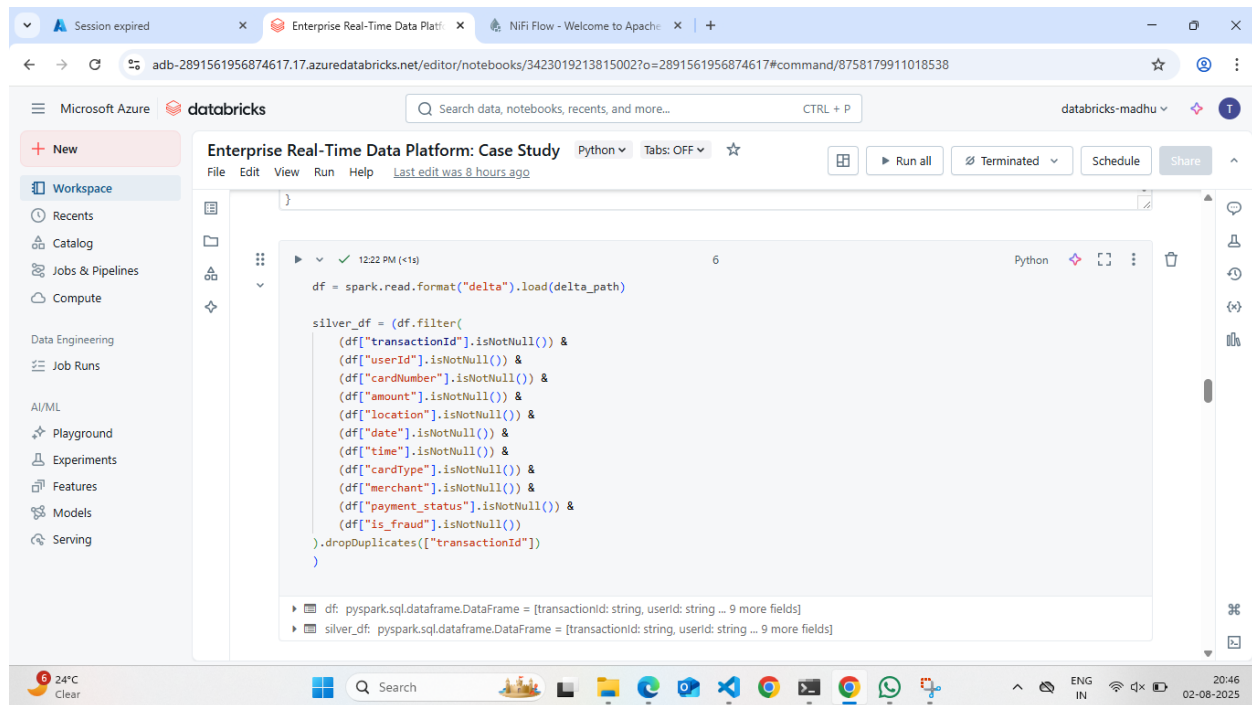


5. Data Transformation and Masking (Silver Layer)

Read Bronze data from Delta Lake. Filtered valid records based on Great Expectations results. Performed PII masking on the cardnumber column: Kept the last 4 digits visible.

Replaced first 12 digits with ***** using a concat and substring function.

Stored the cleaned, masked data in the Delta table: /silver_transactions.



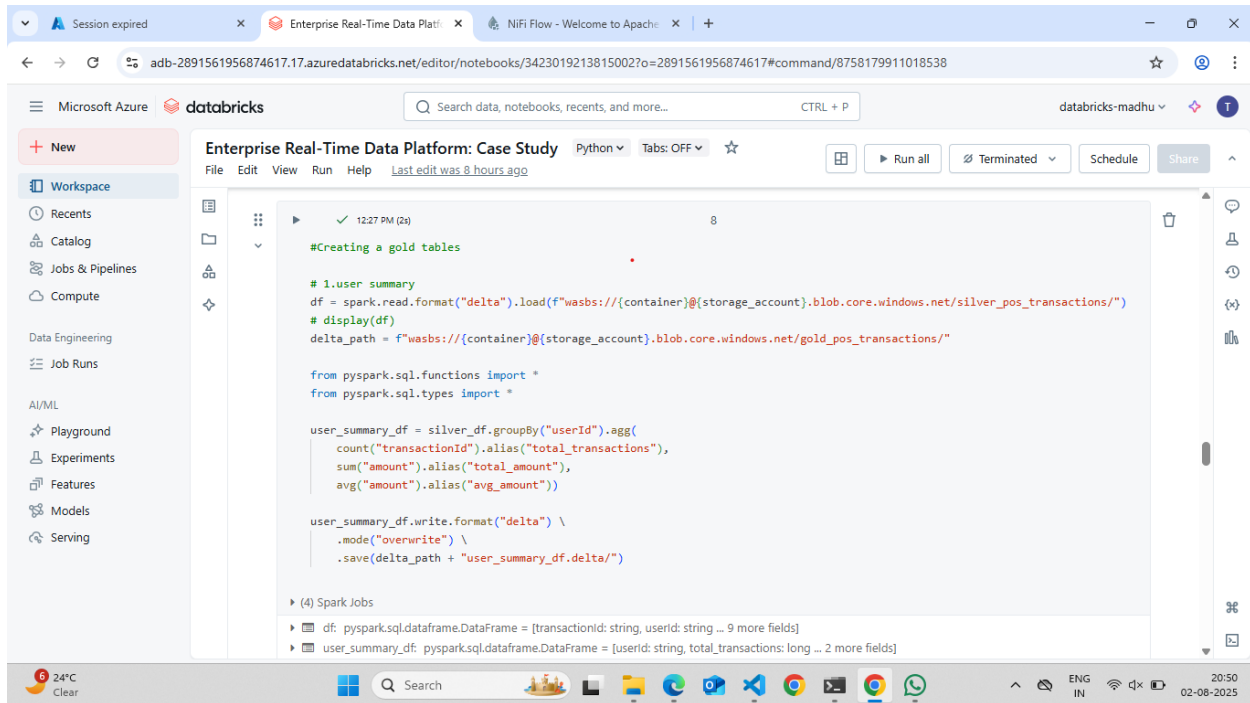
6. Data Aggregation and Analytics (Gold Layer)

Read Silver table data for trusted, cleaned transactions.

Performed aggregation: Grouped data by user_id. Calculated the total amount spent by each customer.

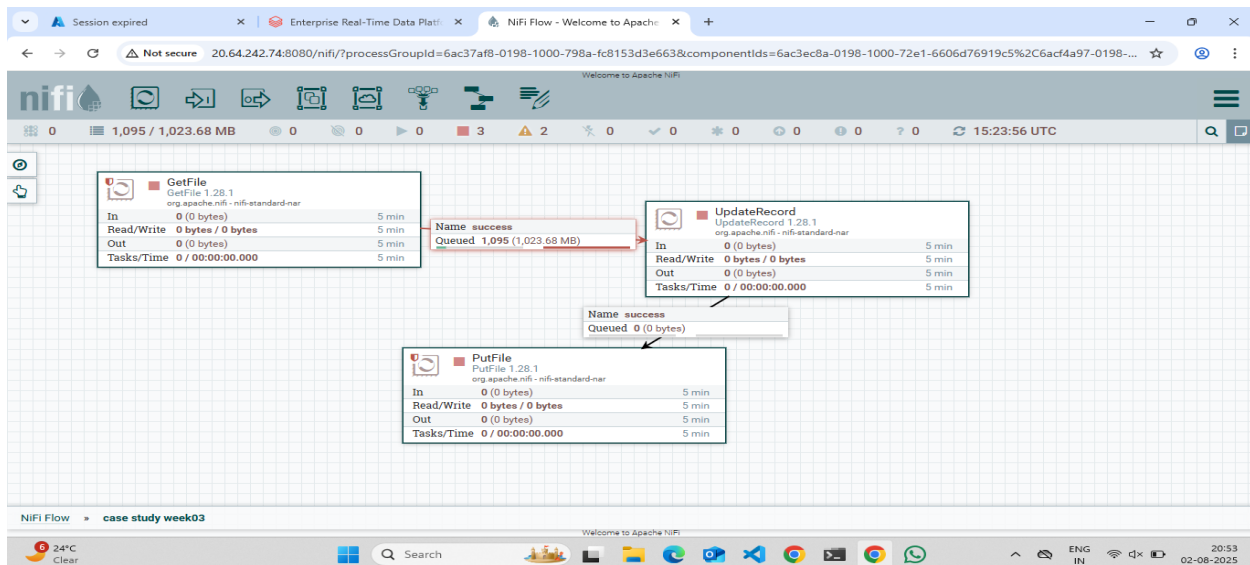
Generated customer-level insights for reporting and downstream analytics.

Stored the aggregated data in the Delta table: /gold/transactions_summary.



7. Data Processing using Apache NiFi

Performed data processing using nifi for masking.



Conclusion:

In this project, I built a complete data pipeline from raw data to final insights. I ingested streaming data from Event Hub using Spark Structured Streaming. I validated the raw data with Great Expectations to ensure data quality. I filtered and masked sensitive information and stored the clean data in Silver tables. I aggregated the Silver data to create Gold tables for business analysis. Finally, I used Apache NiFi to filter the processed data further using the QueryRecord processor and saved the filtered results to files for additional downstream use. This end-to-end flow helps ensure the data is clean, secure, and ready for reporting and analytics.

The screenshot shows the Microsoft Azure portal interface for a container named 'delta-tables'. The left sidebar contains navigation options: Overview, Diagnose and solve problems, Access Control (IAM), Settings, Shared access tokens, Access policy, Properties, and Metadata. The main area displays the container's contents, including a search bar and a table of blobs. The table lists 8 items, showing folders for 'bronze_pos_transactions', 'checkpoints', 'gold_pos_transactions', and 'silver_pos_transactions', along with individual files for each. The table columns are Name, Last modified, Access tier, Blob type, Size, and Lease state.

Name	Last modified	Access tier	Blob type	Size	Lease state
bronze_pos_transactions					
checkpoints					
gold_pos_transactions					
silver_pos_transactions					
bronze_pos_transactions	02/08/2025, 10:27:37	Hot (Inferred)	Block blob	0	Available
checkpoints	02/08/2025, 10:25:12	Hot (Inferred)	Block blob	0	Available
gold_pos_transactions	02/08/2025, 12:27:34	Hot (Inferred)	Block blob	0	Available
silver_pos_transactions	02/08/2025, 12:27:34	Hot (Inferred)	Block blob	0	Available